

JavaScript Async/Await

[Back to JS page](#)

Table of Contents

- [JavaScript Async/Await](#)
 - [The async Keyword](#)
 - [Example 1](#)
 - [The await Keyword](#)
 - [Example 2](#)
 - [Compare Promise and async/await](#)
 - [Example 3](#)
 - [Handling Errors](#)
 - [Example 4](#)
 - [Multiple await](#)
 - [Example 5](#)
 - [Document](#)
 - [Reference](#)

The async Keyword

The `async` keyword makes a function return a Promise.

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Async/Await</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>The async Keyword</h4>
<p id="demo"></p>

<script>
async function myFunction() {
  return "Hello";
}

myFunction().then(function(value) {
  document.getElementById("demo").innerHTML = value;
});
</script>

</body>
</html>
```

Example 1

Result [View Example](#)

The await Keyword

The `await` keyword makes a function wait for a Promise. It can only be used inside an `async` function:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Async/Await</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>The await Keyword</h4>
<p id="demo"></p>

<script>
async function myDisplay() {
  let myPromise = new Promise(function(resolve) {
    setTimeout(function() {
      resolve("Hello after 2 seconds!");
    }, 2000);
  });
  document.getElementById("demo").innerHTML = await myPromise;
}

myDisplay();
</script>

</body>
</html>
```

Example 2

Result [View Example](#)

Compare Promise and async/await

Both approaches work with Promises, but async/await provides cleaner syntax:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Async/Await</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Compare Promise and async/await</h4>
<p id="demo1"></p>
<p id="demo2"></p>

<script>
function getData() {
  return new Promise(function(resolve) {
    setTimeout(function() {
      resolve("Data loaded");
    }, 1000);
  });
}

// Promise version
getData().then(function(value) {
  document.getElementById("demo1").innerHTML = "Promise: " + value;
});

// Async/await version
async function loadData() {
  let value = await getData();
  document.getElementById("demo2").innerHTML = "Async/await: " + value;
}
loadData();
</script>

</body>
</html>
```

Example 3

Result [View Example](#)

Handling Errors

Use `try/catch` to handle errors in async/await:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Async/Await</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Handling Errors</h4>
<p id="demo"></p>

<script>
async function fetchData() {
  try {
    let result = await new Promise(function(resolve, reject) {
      let success = false;
      if (success) {
        resolve("Success!");
      } else {
        reject("Something went wrong");
      }
    });
    document.getElementById("demo").innerHTML = result;
  } catch(error) {
    document.getElementById("demo").innerHTML = "Error caught: " + error;
  }
}

fetchData();
</script>

</body>
</html>
```

Example 4

Result [View Example](#)

Multiple await

You can use multiple `await` statements in sequence:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Async/Await</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Multiple await</h4>
<p id="demo"></p>

<script>
async function processSteps() {
  let step1 = await new Promise(function(resolve) {
    setTimeout(function() {
      resolve("Step 1 done");
    }, 1000);
  });
  document.getElementById("demo").innerHTML += step1 + "<br>";

  let step2 = await new Promise(function(resolve) {
    setTimeout(function() {
      resolve("Step 2 done");
    }, 1000);
  });
  document.getElementById("demo").innerHTML += step2 + "<br>";

  let step3 = await new Promise(function(resolve) {
    setTimeout(function() {
      resolve("Step 3 done");
    }, 1000);
  });
  document.getElementById("demo").innerHTML += step3 + "<br>";

  document.getElementById("demo").innerHTML += "All steps complete!";
}

processSteps();
</script>

</body>
</html>
```

Example 5

Result [View Example](#)

Document

Document in project

You can [Download PDF](#) file.

Reference

- [W3Schools JavaScript Async/Await](#)